OPERATING SYSTEMS ASSIGNMENT 3 – CO3

CO3 Coverage: Evaluate memory-management mechanisms such as linking, paging and segmentation to determine their effectiveness for different system requirements – linking and loading, paging, segmentation, virtual memory, demand paging, page-replacement techniques and disk management.

Nature of Assignment: Individual
Duration: 1 Week
Total Marks: 50
Submission Date: _____

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level)

Mandatory Execution and GitHub Submission

- **Every student must individually execute every task in this assignment**, including all "Complete the Code" programs.
- Create a GitHub repository named **CSA04-<RegisterNumber>-Assignment3** and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C source files, shell scripts, configuration files (Dockerfile, YAML), command outputs and screenshots.
- Include a README.md with your name, register number, compiler and version (for example gcc 13), the Linux distribution and version used, the exact gcc command used to compile (for example gcc -std=c11 -Wall -Wextra -pthread), the steps to run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. **An assignment submitted without a working GitHub link will not be evaluated.**

GitHub Repository Link: _____

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output (or result) with the expected output.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Assignment Principle

Every question asks you to design, implement, analyse and evaluate the solution yourself, so that you demonstrate evidence of:

Design → Implementation → Analysis → Evaluation

Q1 – Linking, Loading and Address Binding — 6 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Updating a Banking Application

A bank's branch software is built from many modules and libraries. When a security fix is made to the encryption library, every branch must currently receive a new 200 MB executable.

Implementation Tasks You must:

- Build the same program with static linking and with dynamic (shared) libraries
- Compare the sizes of the executables and list the shared libraries with ldd
- Load a library at run time with dlopen() and dlsym()
- Show the addresses at which the program and its libraries are loaded (/proc/self/maps)

Evaluation You must:

- Explain compile-time, load-time and execution-time address binding
- Explain the roles of the linker and the loader
- Evaluate static and dynamic linking for memory use, start-up time and ease of updating • Explain position-independent code

Experimental Investigation You must:

- Measure the start-up time and memory use of both versions
- Replace the shared library without recompiling the program and show the change
- Run two copies and show which pages are shared
- Test what happens when the shared library is missing

Conclusion

Which linking method should the bank use, and why?

Alternative Concept and Complexity Analysis

If the bank loaded the encryption library at run time with dlopen() only when needed, instead of linking it dynamically at start-up, what would the solution be? Evaluate the start-up time, the memory used when encryption is not needed, and the error handling needed if the library is missing.

Q2 – Paging — 8 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Memory for a Multi-User College Server

A college server runs hundreds of student programs of very different sizes. Allocating each program one contiguous block of memory left many small unusable gaps, and large programs could not be loaded although enough total memory was free.

Implementation Tasks You

must:

- Explain external fragmentation using a diagram of the contiguous allocation
- Translate logical addresses to physical addresses with a page table
- Calculate the size of the page table for a 32-bit and a 48-bit address space with 4 KB pages
- Calculate the effective memory-access time with and without a TLB

Evaluation You

must:

- Explain how paging removes external fragmentation and why internal fragmentation remains
- Evaluate page sizes of 1 KB, 4 KB and 2 MB
- Explain hierarchical (multi-level), hashed and inverted page tables
- Explain how protection and shared pages are provided with paging

Experimental Investigation You

must:

- Calculate the internal fragmentation for 100 processes of random sizes with each page size
- Calculate the effective access time for TLB hit ratios of 80%, 95% and 99%
- Find the page size of your Linux system (getconf PAGESIZE) and the page faults of a program (/usr/bin/time -v)
- Test addresses at the start and end of a page and beyond the last page

Conclusion

What page size would you choose for the college server, and why?

Complete the Code

Complete TODO 1 below and execute the program.

```
#include <stdio.h>

#define PAGE_SIZE 1024
#define PAGES 8

/* Page table of a process: frame number, or -1 if the page is not in memory. */ int
page_table[PAGES] = {5, 2, -1, 7, 0, -1, 3, -1};

int main(void)
{
    unsigned logical[] = {0, 1500, 3100, 6200, 9000, 2049};
for (int i = 0; i < 6; i++) {         unsigned a =
logical[i];
        // TODO 1: Split a into page number p = a / PAGE_SIZE and offset d = a %
        //   PAGE_SIZE. If p is outside the table print "invalid address"; if the frame
//   is -1 print "page fault"; otherwise print the physical address frame *         //
PAGE_SIZE + d

    }
    return 0;
}
```

Expected output:

```
   0 -> page 0 offset    0 : physical 5120
1500 -> page 1 offset  476 : physical 2524
3100 -> page 3 offset   28 : physical 7196
6200 -> page 6 offset   56 : physical 3128
9000 -> page 8 offset  808 : invalid address
2049 -> page 2 offset    1 : page fault
```

Change PAGE_SIZE to 2048 and recalculate the results by hand before running the program again.

Alternative Concept and Complexity Analysis

If the server used an inverted page table (one entry per physical frame) instead of a per-process multi-level page table, what would the solution be? Evaluate the space used (proportional to physical memory versus to the address space of each process) and the time to translate an address (a hash look-up versus a walk of several levels).

Q3 – Segmentation **6 Marks**

Bloom's Level: K5 – Evaluate

Problem Context – Student Programs with Separate Code, Stack and Data

Students' programs have code that should be read-only and shared by everyone running the same compiler, a stack that grows, and data of varying size. Paging alone treats all of these the same.

Implementation Tasks You

must:

- Translate (segment, offset) addresses using a segment table with base and limit
- Show what happens when an offset exceeds the segment limit
- Draw how segmentation with paging (as in the Intel x86 architecture) translates an address
- Show the segments of a running program on Linux (/proc/<pid>/maps)

Evaluation You

must:

- Explain how segmentation matches the programmer's view of memory
- Explain how segments are protected and shared
- Evaluate the external fragmentation caused by segmentation
- Compare pure paging, pure segmentation and segmented paging

Experimental Investigation You

must:

- Translate 8 addresses by hand and verify them with a small program
- Trigger a segmentation fault and explain what the OS did
- Calculate the fragmentation after loading and removing 20 segments of random sizes • Compare the size of the tables needed in each scheme

Conclusion

Why do modern operating systems rely mainly on paging?

Alternative Concept and Complexity Analysis

If the programs used pure paging instead of segmentation, what would the solution be? Evaluate the fragmentation (internal versus external), the size of the tables, the time to translate an address, and how easily code is shared and protected.

Q4 – Virtual Memory and Demand Paging **8 Marks**

Bloom's Level: K5 – Evaluate

Problem Context – Running Large Medical-Imaging Software on a Laptop

A radiologist needs to run an image-processing program that uses 24 GB of memory on a laptop with 16 GB of RAM.

Implementation Tasks You

must:

- Explain the steps of handling a page fault
- Calculate the effective access time for page-fault rates of 1 in 1,000, 1 in 100,000 and 1 in 1,000,000 (memory 100 ns, fault service 8 ms)
- Explain copy-on-write and how it makes fork() fast
- Explain thrashing and the working-set model

Evaluation You

must:

- Explain how demand paging lets a 24 GB program run in 16 GB
- Evaluate the maximum acceptable page-fault rate for a 10% slow-down
- Explain how the number of frames per process should be decided (equal and proportional allocation)
- Explain how the OS detects and stops thrashing

Experimental Investigation You

must:

- Write a program that touches memory in sequential and random order and count its page faults
- Measure the run time as the memory used grows beyond the free RAM (use a virtual machine or a memory limit)
- Show copy-on-write with fork() by measuring memory before and after writing
- Plot the page faults against the number of frames for a reference string

Conclusion

Is buying more RAM or improving the program's locality the better solution for the radiologist? Justify.

Alternative Concept and Complexity Analysis

If the radiologist's program were run on a machine with 32 GB of RAM instead of relying on demand paging, what would the solution be? Evaluate the effective access time in both cases for the measured page-fault rate, and the cost of extra memory against the cost of the slower run.

Q5 – Page-Replacement Techniques **8 Marks**

Bloom's Level: K5 – Evaluate

Problem Context – Choosing a Policy for a Web Server

A web server keeps frequently used pages of data in memory. When memory is full, the OS must choose which page to remove, and a poor choice causes slow responses.

Implementation Tasks You must:

- Implement FIFO, LRU and Optimal page replacement
- Count the page faults for a given reference string with 3 and 4 frames
- Demonstrate Belady's anomaly with FIFO
- Explain the clock (second-chance) algorithm as a practical approximation of LRU

Evaluation You must:

- Explain why Optimal is the best possible but cannot be implemented
- Explain why LRU never suffers from Belady's anomaly (the stack property)
- Evaluate the hardware support LRU needs and how the clock algorithm avoids it • Compare the algorithms for a workload with strong locality

Experimental Investigation You must:

- Generate reference strings with strong locality and with random access, and count the faults of each algorithm for 3 to 10 frames
- Plot the faults against the number of frames for each algorithm
- Measure the running time of each algorithm for 100,000 references
- Verify your program's results with a hand calculation for one string

Engineering Decision

Which page-replacement policy would you recommend for the web server? Use your measurements.

Complete the Code

Complete TODO 1 below to finish LRU, and execute the program.

```
#include <stdio.h>

#define MAXF 8

int find(const int f[], int n, int page)
{
    for (int i = 0; i < n; i++) if (f[i] == page) return i;
return -1; }

int fifo(const int ref[], int len, int frames)
{
    int f[MAXF], used = 0, next = 0, faults = 0;
for (int i = 0; i < len; i++) {         if
(find(f, used, ref[i]) >= 0) continue;
faults++;
        if (used < frames) f[used++] = ref[i];         else {
f[next] = ref[i]; next = (next + 1) % frames; }
    }
    return faults; }

int lru(const int ref[], int len, int frames)
{
    int f[MAXF], last[MAXF], used = 0, faults = 0;
for (int i = 0; i < len; i++) {         int k =
find(f, used, ref[i]);        if (k >= 0) {
last[k] = i; continue; }         faults++;
        // TODO 1: If a frame is free use it; otherwise replace the frame whose last use
//   (last[]) is the oldest. Record last[] = i for the frame used

    }
    return faults; }

int optimal(const int ref[], int len, int frames)
{
    int f[MAXF], used = 0, faults = 0;     for (int i = 0; i <
len; i++) {         if (find(f, used, ref[i]) >= 0) continue;
faults++;         if (used < frames) { f[used++] = ref[i];
continue; }         int victim = 0, farthest = -1;         for
(int j = 0; j < frames; j++) {
            int next = len;                  /* never used again */
            for (int t = i + 1; t < len; t++) if (ref[t] == f[j]) { next = t; break; }
if (next > farthest) { farthest = next; victim = j; }
        }
        f[victim] = ref[i];
    }
    return faults; }

int main(void)
{
    int ref[] = {7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1};
int len = 20;
    for (int fr = 3; fr <= 4; fr++)         printf("%d frames: FIFO %d, LRU %d,
Optimal %d\n", fr, fifo(ref, len, fr),                lru(ref, len, fr),
optimal(ref, len, fr));     int b[] = {1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5};
printf("Belady string, FIFO: 3 frames %d faults, 4 frames %d faults\n",
fifo(b, 12, 3), fifo(b, 12, 4));     return 0;
}
```

Expected output:

```
3 frames: FIFO 15, LRU 12, Optimal 9
4 frames: FIFO 10, LRU 8, Optimal 8
```

```
Belady string, FIFO: 3 frames 9 faults, 4 frames 10 faults
```

Explain why FIFO has more faults with 4 frames than with 3 for the second reference string, and why LRU cannot behave like this.

Alternative Concept and Complexity Analysis

If the clock (second-chance) algorithm were used instead of exact LRU, what would the solution be? Evaluate the time per reference (updating a time stamp or a list on every access versus setting one bit), the extra space per frame, and how close its page faults come to LRU in your simulation.

Q6 – Disk Management **4 Marks**

Bloom's Level: K5 – Evaluate

Problem Context – Storage for a Hospital Records Server

The hospital records server must start reliably, survive the failure of a disk and have enough swap space for peak load.

Implementation Tasks You

must:

- Explain low-level formatting, partitioning and logical formatting
- Explain the boot block and the boot process
- Explain how bad blocks are handled
- Decide the size and location of the swap space

Evaluation You

must:

- Compare RAID 0, 1, 5 and 10 for reliability, capacity and speed
- Choose the RAID level for the records server
- Evaluate swap on a separate disk versus a swap file
- Explain how disk partitions are viewed on Linux (lsblk, fdisk -l)

Experimental Investigation You

must:

- Show the partitions and swap of your Linux system
- Calculate the usable capacity of 6 disks of 4 TB at each RAID level
- Calculate the probability of data loss for RAID 1 and RAID 5 given a disk failure rate
- Measure the read speed of a file on SSD and hard disk if both are available

Conclusion

Which storage configuration would you recommend for the records server, and why?

Alternative Concept and Complexity Analysis

If the records server used RAID 6 instead of RAID 5 (or RAID 10), what would the solution be? Evaluate the usable capacity of 6 disks of 4 TB, the time to write a block (the extra parity calculation), and the number of disk failures that can be survived.

Q7 – Design a Memory-Management Plan from First Principles **10 Marks**

Bloom's Level: K5 – Evaluate

Problem Context – Memory Management for an Edge Device in a Smart Factory

A smart factory installs edge devices with 2 GB of RAM and no swap disk. Each device runs a real-time control task that must never be delayed by a page fault, a machine-learning task that needs 1.5 GB, and a logging task. The vendor wants to add more tasks in the future.

You are not given the design. You must develop it from the requirements.

Stage 1 – Interpret the Problem | 2 Marks

- Identify the memory needs and timing constraints of each task
- Identify which tasks can tolerate page faults and which cannot
- State the requirements as measurable statements

Stage 2 – Compare Candidate Mechanisms | 3 Marks

- Compare contiguous allocation, paging, segmentation and demand paging for each task
- Decide how memory is locked for the real-time task (for example mlock()) • Present a comparison table with justification

Stage 3 – Implement and Verify | 3 Marks

- Write a program that locks the memory of a time-critical loop and measure its worst-case delay with and without locking
- Simulate the memory use of the three tasks with a page-replacement simulator • Verify that the real-time task is never delayed

Stage 4 – Defend the Design | 2 Marks

- Justify your plan using your measurements
- Explain what changes if a fourth task of 500 MB is added
- Identify one remaining limitation

Alternative Concept and Complexity Analysis

If the edge device used a real-time operating system with no virtual memory instead of Linux with locked memory for the control task, what would the solution be? Evaluate the worst-case delay of the control task, the memory available to the machine-learning task, and the effort of adding new tasks.

MARK DISTRIBUTION

Question	Assessed Area	Bloom's Level	Marks
Q1	Linking, loading and address binding	K5 – Evaluate	6
Q2	Paging	K5 – Evaluate	8
Q3	Segmentation	K5 – Evaluate	6
Q4	Virtual memory and demand paging	K5 – Evaluate	8
Q5	Page-replacement techniques	K5 – Evaluate	8
Q6	Disk management	K5 – Evaluate	4
Q7	Memory-management plan from first principles	K5 – Evaluate	10
	TOTAL		**50**

Submission Requirements

GitHub repository link (mandatory) containing the executed code for every task, including every completed "Complete the Code" program, script and configuration file, with input files, output screenshots and a README.md.

- Your report containing the required evidence for Q1–Q7
- C source code for all programs
- Address-translation workings, graphs, measurements and command outputs

Assessment Verification

Students may be asked to explain, execute or modify any part of their design, code, calculations or results during a brief verification discussion/viva. The student should be able to explain and defend the work submitted as their own.

TABLE OF CONTENTS

Q1 – Linking, Loading and Address Binding

Q2 – Paging

Q3 – Segmentation

Q4 – Virtual Memory and Demand Paging

Q5 – Page-Replacement Techniques

Q6 – Disk Management

Q7 – Memory-Management Plan from First Principles

Overall Conclusion

References

Q1 – LINKING, LOADING AND ADDRESS BINDING (6 MARKS)

1. Introduction

A banking application is built from several modules and libraries. When an encryption library receives a security update, static linking may require distributing a newly built executable to every branch. Dynamic linking allows a program to use a shared library, which can be updated separately when compatibility is maintained.

2. Linking and Loading

The linker combines object files and libraries, resolves external symbols and produces an executable. The loader is part of the operating system. It creates the process address space, maps the executable and required shared libraries, performs required relocations and transfers control to the program entry point.

3. Address Binding

Binding type	Meaning	Evaluation
Compile-time	Addresses are fixed when the program is compiled, when the final memory location is known.	Fast address use, but relocating the program may require recompilation.
Load-time	The compiler produces relocatable code; the loader adjusts addresses when the program is loaded.	Allows the program to be placed at different memory locations.
Execution-time	Address translation is performed while the program runs, normally with hardware support such as the MMU.	Most flexible; supports virtual memory and process relocation.

4. Static Linking and Dynamic Linking

Factor	Static linking	Dynamic linking
Executable size	Usually larger because required library code is included.	Usually smaller because library code is stored separately.
Memory use	Separate executable copies can contain repeated library code.	Read-only shared library pages may be shared by processes.
Start-up	May avoid resolving external shared libraries at start-up.	The loader must map and resolve required shared libraries.
Updating	A library fix commonly requires rebuilding and redistributing the executable.	A compatible shared library can be updated separately.

Dependency	Fewer run-time library dependencies.	Program depends on the required shared library being available.

5. Position-Independent Code (PIC)

Position-independent code can execute correctly regardless of the address at which it is loaded. Shared libraries commonly use PIC so that one library can be mapped at different virtual addresses in different processes without rewriting every instruction.

6. Practical Linux Commands

```
# Build a shared library
gcc -fPIC -c encrypt.c -o encrypt.o
gcc -shared -o libencrypt.so encrypt.o

# Build a dynamically linked application
gcc bank.c -L. -lencrypt -Wl,-rpath,'$ORIGIN' -o bank_dynamic

# Build a statically linked application (requires static library)
gcc bank.c ./libencrypt.a -o bank_static

# Compare executable sizes and dependencies
ls -lh bank_dynamic bank_static
ldd bank_dynamic

# Inspect mapped executable and library regions
cat /proc/$(pidof bank_dynamic)/maps
```

If a static library is not available on the Linux installation, the static build command may fail. Record the exact error and install the relevant development package or state that the environment does not provide the static archive.

7. Run-Time Loading with dlopen() and dlsym()

```
#include <stdio.h>
#include <dlfcn.h>

int main(void)
{
    void *handle = dlopen("./libencrypt.so", RTLD_NOW);
    if (!handle) {
        fprintf(stderr, "%s\n", dlerror());
        return 1;
    }

    dlerror(); /* clear previous error */
    void (*encrypt_message)(void) =
        (void (*)(void)) dlsym(handle, "encrypt_message");

    const char *error = dlerror();
    if (error != NULL) {
        fprintf(stderr, "%s\n", error);
        dlclose(handle);
        return 1;
    }
```

```
    encrypt_message();
    dlclose(handle);
    return 0;
}
```

Compile the run-time loading example with: gcc runtime_load.c -o runtime_load -ldl. The function name and function signature must match the symbol exported by the supplied shared library.

8. Evaluation and Recommendation

For a bank that frequently patches a common encryption library across many branches, dynamic linking is generally more convenient because the library can be updated independently. The bank must manage library versions, compatibility and secure deployment. Static linking is appropriate when a self-contained executable is more important than independent library updates.

9. Alternative: Load the Library Only When Needed

Using dlopen() only when encryption is required can reduce start-up work and avoid mapping the library in executions that never use encryption. The application must handle a missing library, a missing symbol and an incompatible library version. The start-up benefit and memory saving depend on how often encryption is used.

10. Evidence to Insert

- Screenshot of successful static and dynamic builds.
- Output of ls -lh and ldd.
- Output showing dlopen()/dlsym() success and error handling.
- Relevant /proc/<pid>/maps lines.
- Measured start-up time and memory use for both versions.

Measurement table (fill with actual results):

Version	Executable size	Start-up time	Memory use	Notes
Static	_____	_____	_____	_____
Dynamic	_____	_____	_____	_____

Q2 – PAGING (8 MARKS)

1. Introduction

Paging divides a process's logical address space into fixed-size pages and physical memory into equal-size frames. Any page can be placed in any available frame. A page table maps each virtual page number to a physical frame number.

2. External and Internal Fragmentation

In contiguous allocation, a process needs one continuous free region. Free memory may be split into small gaps, producing external fragmentation even when the total free memory is sufficient. Paging removes the requirement for a process to occupy one contiguous physical region. However, the final page of a process may not be completely filled, causing internal fragmentation.

Diagram to draw/paste: show physical memory as several occupied blocks separated by small free holes for contiguous allocation. Below it, show a process divided into pages placed in non-adjacent frames.

3. Logical-to-Physical Address Translation

For page size S, a logical address A is divided into page number p = A / S and offset d = A % S. The page table gives the frame number f. If the page is present, physical address = f × S + d. If the page is not present, a page fault occurs.

Logical address	Page number	Offset	Frame	Physical address / result
0	0	0	5	5120
1500	1	476	2	2524
3100	3	28	7	7196
6200	6	56	3	3128
9000	8	808	Outside table	Invalid address
2049	2	1	Not present	Page fault

4. Page Table Size

With 4 KB pages, page size = 4 × 1024 = 4096 bytes = 2^12. Assuming a simple single-level page table and 4 bytes per page-table entry:

Virtual address space	Number of pages / entries	Page-table size
32-bit	2^(32−12) = 2^20 = 1,048,576	1,048,576 × 4 = 4,194,304 bytes = 4 MiB
48-bit	2^(48−12) = 2^36	2^36 × 4 = 2^38 bytes = 256 GiB

A 48-bit address space makes a flat page table impractically large. Real systems use multi-level or other page-table structures so that only needed portions are allocated.

5. Effective Access Time with a TLB

A Translation Lookaside Buffer (TLB) caches recent page-table translations. Let memory access time be m and TLB hit ratio be α. With a one-level page table and serial lookup, an illustrative effective access time is EAT = α(m) + (1−α)(2m), ignoring the TLB lookup time.

TLB hit ratio	Illustrative EAT when m = 100 ns
80%	0.8(100) + 0.2(200) = 120 ns
95%	0.95(100) + 0.05(200) = 105 ns
99%	0.99(100) + 0.01(200) = 101 ns

6. Page-Size Evaluation

Page size	Advantages	Limitations
1 KB	Less internal fragmentation for small processes.	More pages and larger page tables; more translation entries may be needed.
4 KB	Common compromise between page-table overhead and internal fragmentation.	May still waste part of the final page.
2 MB	Fewer page-table entries and useful for large contiguous working sets.	Can waste substantial memory for small allocations; requires suitable alignment and support.

7. Page-Table Structures and Protection

- Hierarchical page tables divide the virtual address into fields and use multiple levels. Only the required lower-level tables need to exist.
- Hashed page tables hash virtual page numbers to entries, which is useful for large address spaces.

- Inverted page tables maintain an entry for each physical frame rather than a full entry for every virtual page.
- Protection bits can mark pages as read-only, writable, executable or user-accessible. Shared pages can map the same physical frame into more than one process.

8. Complete the Paging Address-Translation Program

```
#include <stdio.h>
#define PAGE_SIZE 1024
#define PAGES 8
int page_table[PAGES] = {5, 2, -1, 7, 0, -1, 3, -1};

int main(void)
{
    unsigned logical[] = {0, 1500, 3100, 6200, 9000, 2049};
    for (int i = 0; i < 6; i++) {
        unsigned a = logical[i];
        unsigned p = a / PAGE_SIZE;
        unsigned d = a % PAGE_SIZE;

        if (p >= PAGES) {
            printf("%5u -> page %u offset %6u : invalid address\n",
                   a, p, d);
        } else if (page_table[p] == -1) {
            printf("%5u -> page %u offset %6u : page fault\n",
                   a, p, d);
        } else {
            unsigned physical =
                (unsigned)page_table[p] * PAGE_SIZE + d;
            printf("%5u -> page %u offset %6u : physical %u\n",
                   a, p, d, physical);
        }
    }
    return 0;
}
```

Compile: gcc -std=c11 -Wall -Wextra paging.c -o paging. Run: ./paging. The output should match the assignment's expected output for PAGE_SIZE = 1024. Change PAGE_SIZE to 2048 and independently recalculate the page numbers, offsets and results.

9. Experimental Investigation and Decision

- Generate 100 process sizes and calculate internal fragmentation as allocated page space minus requested process size.
- Calculate EAT for TLB hit ratios of 80%, 95% and 99%.
- Run getconf PAGESIZE to identify the system page size.
- Run /usr/bin/time -v ./program and record major and minor page faults.
- Test addresses at a page boundary, at the final valid page and beyond the page table.

For a general-purpose college server, 4 KB pages are a reasonable baseline because they balance internal fragmentation and page-table overhead. Large pages may be beneficial for selected large-memory workloads after measurement.

Q3 – SEGMENTATION (6 MARKS)

1. Introduction

Segmentation divides a program into logical units such as code, data, stack and heap. Each segment has a base address and a limit. A logical address contains a segment number and an offset within that segment.

2. Address Translation

For segment s and offset d, the operating system or hardware checks whether s is a valid segment and whether d is smaller than the segment limit. If valid, physical address = base[s] + d. If the offset is outside the limit, a protection fault is raised.

Segment	Base	Limit	Example offset	Result
Code	1000	500	120	1120 – valid
Data	4000	800	700	4700 – valid
Stack	8000	600	650	Invalid – exceeds limit
Heap	10000	1200	100	10100 – valid

3. Example Program for Segment Translation

```
#include <stdio.h>

#define N 4
int base[N]  = {1000, 4000, 8000, 10000};
int limit[N] = {500, 800, 600, 1200};

int main(void)
{
    int seg[] = {0, 1, 2, 3, 2, 1, 0, 3};
    int off[] = {100, 700, 650, 100, 599, 800, 499, 1200};

    for (int i = 0; i < 8; i++) {
        int s = seg[i], d = off[i];
        if (s < 0 || s >= N || d < 0 || d >= limit[s])
            printf("segment %d offset %d: protection fault\n", s, d);
        else
            printf("segment %d offset %d: physical %d\n",
                   s, d, base[s] + d);
    }
    return 0;
}
```

Compile and run with gcc -std=c11 -Wall -Wextra segment.c -o segment && ./segment. The table values are illustrative for demonstrating the translation rule.

4. Segmentation with Paging

Segmentation with paging combines the logical view of segments with fixed-size pages. A logical address first identifies a segment and an offset. The segment table identifies the page table for that segment. The offset is divided into a page number and a page offset; the page table then identifies the physical frame.

Diagram to draw/paste: Logical address [segment number | offset] → segment table → page table → frame number + page offset → physical address.

5. Comparison of Memory Schemes

Method	Fragmentation	Programmer view	Evaluation
Pure paging	No external fragmentation; internal fragmentation possible.	Memory is divided into equal pages.	Efficient allocation and virtual memory support.
Pure segmentation	External fragmentation can occur; segment sizes vary.	Matches code, data and stack logical units.	Good logical protection and sharing, but allocation is more complex.
Segmented paging	Avoids external fragmentation within physical allocation; internal fragmentation remains at page boundaries.	Preserves segments while paging their contents.	Combines logical organisation with flexible physical allocation.

6. Protection, Sharing and Linux Evidence

- Code segments can be read-only and executable, while data segments can be writable and non-executable.
- Shared libraries can be mapped into multiple processes, allowing shared read-only code pages.
- Segment limits can detect an offset outside the permitted region.
- Modern Linux processes primarily use a flat virtual address space with paging; /proc/<pid>/maps displays mapped regions and permissions.

```
# Find the process ID
pidof <program_name>

# Display mapped regions (replace PID)
cat /proc/<pid>/maps

# Trigger a controlled invalid memory access only in a test program
# Observe the segmentation-fault message and exit status
```

7. Experimental Investigation and Evaluation

- Translate eight (segment, offset) addresses by hand and compare them with the program.
- Run a controlled test that accesses invalid memory and explain that the OS terminates or signals the process (commonly SIGSEGV).
- Simulate loading and removing 20 randomly sized segments; record free holes and total external fragmentation.
- Compare the storage required for segment tables, page tables and combined structures.

Modern general-purpose operating systems rely mainly on paging because it supports virtual memory, avoids external fragmentation in physical allocation and works efficiently with hardware address translation. Logical protection and sharing can still be provided through page permissions and mappings.

Q4 – VIRTUAL MEMORY AND DEMAND PAGING (8 MARKS)

1. Introduction

Virtual memory allows a process to use a virtual address space larger than the physical RAM available. Demand paging loads a page into RAM only when the process first accesses it. Pages not currently needed can remain in backing storage.

2. Page-Fault Handling

1. The CPU references a virtual page that is not currently present in RAM.
2. The MMU detects the missing page and raises a page-fault exception.
3. The operating system checks whether the address is valid and whether the process has permission to access it.
4. If the reference is invalid, the OS reports an error to the process.
5. If valid, the OS finds a free frame or selects a victim page for replacement.
6. The required page is read from backing storage into the selected frame.
7. The page table and TLB information are updated.
8. The instruction that caused the fault is restarted.

3. Effective Access Time Calculation

Given memory access time = 100 ns and page-fault service time = 8 ms = 8,000,000 ns. Using EAT = (1 − p) × 100 + p × 8,000,000 ns, where p is the page-fault probability:

Fault rate	Calculation	EAT
1 in 1,000	0.999(100) + 0.001(8,000,000)	8,099.9 ns
1 in 100,000	0.99999(100) + 0.00001(8,000,000)	179.999 ns ≈ 180 ns
1 in 1,000,000	0.999999(100) + 0.000001(8,000,000)	107.9999 ns ≈ 108 ns

The calculation is illustrative and treats the page-fault service time as the additional service cost for a faulting access, as specified by the assignment.

4. Demand Paging for a 24 GB Program on a 16 GB Laptop

The program does not need all 24 GB to be resident in RAM simultaneously. Demand paging keeps the currently active pages in physical memory and leaves inactive pages in backing storage. The

program can run if its working set and the operating system's other memory needs fit within available RAM and backing storage is sufficient. If the active working set is larger than available memory, frequent page faults and thrashing may make the program unusably slow.

5. Maximum Page-Fault Rate for 10% Slowdown

For a maximum 10% slowdown, target EAT ≤ 110 ns when normal memory access is 100 ns. Solve: $(1 - p)100 + p(8{,}000{,}000) \leq 110$. Thus $p(7{,}999{,}900) \leq 10$ and $p \leq 10 / 7{,}999{,}900 \approx 0.00000125$, approximately one fault per 800,000 references. This is a simplified estimate based on the given service time.

6. Copy-on-Write and fork()

Copy-on-write allows parent and child processes to initially share physical pages after fork(). The pages are marked so that a write triggers a protection fault. The operating system then creates a private copy for the writing process. This avoids copying every page immediately and makes fork() faster when most pages are not modified.

7. Thrashing and Working Set

Thrashing occurs when processes spend most of their time servicing page faults instead of executing useful instructions. The working-set model estimates the pages a process has used during a recent window. The OS can use working-set information and page-fault frequency to allocate more frames, reduce the degree of multiprogramming or suspend a process.

8. Equal and Proportional Frame Allocation

Method	How frames are assigned	Evaluation
Equal allocation	Each process receives approximately the same number of frames.	Simple, but ignores differences in process size and working set.
Proportional allocation	Frames are assigned according to each process's size or memory demand.	Uses memory more appropriately when process sizes differ.

9. Experimental Investigation

- Write a memory-touching program that accesses allocated memory sequentially and randomly; record page faults and execution time.

- Increase memory use beyond available free RAM only in a controlled virtual machine or under a safe memory limit.
- Demonstrate copy-on-write by recording memory use before and after a child writes to shared pages.
- Plot page faults against the number of frames for a chosen reference string.

10. Recommendation

The best first step is to improve locality and reduce unnecessary memory use, because a program with a smaller working set can run efficiently without constant disk access. More RAM is appropriate when the measured working set genuinely exceeds available memory. The decision should compare hardware cost with the measured performance loss caused by paging.

Q5 – PAGE-REPLACEMENT TECHNIQUES (8 MARKS)

1. Introduction

When a page fault occurs and no free frame is available, the OS must select a resident page to remove. FIFO removes the page that entered memory earliest. LRU removes the page that has not been used for the longest time. Optimal removes the page whose next use is farthest in the future.

2. Algorithm Comparison

Algorithm	Replacement rule	Strength	Limitation
FIFO	Replace the oldest loaded page.	Simple queue implementation.	Can suffer from Belady's anomaly.
LRU	Replace the least recently used page.	Uses recent history; has the stack property.	Exact implementation requires tracking access history.
Optimal	Replace the page whose next use is farthest in the future.	Minimum possible faults for a known reference string.	Cannot be implemented online because future references are unknown.
Clock / Second Chance	Use a circular pointer and reference bit; give recently used pages another chance.	Low overhead approximation to LRU.	May not match exact LRU fault count.

3. LRU Code Completion

```
int lru(const int ref[], int len, int frames)
{
    int f[MAXF], last[MAXF], used = 0, faults = 0;

    for (int i = 0; i < len; i++) {
        int k = find(f, used, ref[i]);
        if (k >= 0) {
            last[k] = i;
            continue;
        }

        faults++;
        int victim;
        if (used < frames) {
            victim = used++;
            f[victim] = ref[i];
        } else {
            victim = 0;
            for (int j = 1; j < frames; j++) {
                if (last[j] < last[victim])
                    victim = j;
            }
            f[victim] = ref[i];
        }
        last[victim] = i;
```

```
        }
        return faults;
    }
```

Insert this function body in place of the TODO in the supplied program. Compile using gcc -std=c11 -Wall -Wextra page_replace.c -o page_replace and run ./page_replace.

4. Expected Results from the Assignment Reference String

Frames	FIFO faults	LRU faults	Optimal faults
3	15	12	9
4	10	8	8

For the Belady reference string {1,2,3,4,1,2,5,1,2,3,4,5}, FIFO produces 9 faults with 3 frames and 10 faults with 4 frames. This demonstrates Belady's anomaly: increasing the number of frames can increase FIFO page faults.

5. Why LRU Does Not Suffer from Belady's Anomaly

LRU is a stack algorithm. The set of pages held with n frames is contained in the set held with n+1 frames at the corresponding point in the reference string. Therefore, increasing the number of frames cannot increase the number of LRU page faults.

6. Clock Algorithm

9. Arrange frames in a circular list and maintain a reference bit for each frame.
10. When a page is accessed, set its reference bit to 1.
11. When replacement is needed, inspect the frame under the clock hand.
12. If its reference bit is 1, clear the bit and move to the next frame.
13. If its reference bit is 0, replace that page and advance the hand.

Clock avoids maintaining an exact last-used timestamp for every access. It is usually cheaper than exact LRU and provides a practical approximation.

7. Experimental Investigation

- Generate reference strings with strong locality and random access.
- Run FIFO, LRU and Optimal for 3 to 10 frames and record page faults.
- Plot frame count against page faults for each algorithm.
- Measure each algorithm's execution time for 100,000 references.

- Verify one short reference string manually and compare with program output.

Recommendation: For a general web-server workload with locality, an LRU-like policy or clock/second-chance is preferable to FIFO. Exact LRU can have higher tracking overhead, while clock provides a practical compromise. Use measured fault counts and run time to defend the final selection.

Q6 – DISK MANAGEMENT (4 MARKS)

1. Introduction

Disk management prepares storage devices for use, organises partitions and file systems, supports booting and manages bad blocks and swap space. A hospital records server requires reliability, capacity and predictable recovery.

2. Formatting and Boot Process

Operation	Description
Low-level formatting	Organises the physical storage medium into sectors and structures used by the device.
Partitioning	Divides a disk into logical regions, each of which can be used for a file system, swap or another purpose.
Logical formatting	Creates a file system structure such as directories, metadata and free-space information.
Boot block	Contains or locates the code needed to continue the boot process.

During boot, firmware selects a boot device and starts the bootloader. The bootloader loads the operating-system kernel and required initial data. The kernel initialises hardware and mounts the root file system before starting system services.

3. Bad Blocks and Swap

Bad blocks are sectors that cannot reliably store data. The storage device or operating system can detect and remap defective sectors using spare areas or file-system mechanisms. Swap provides backing storage for memory pages that are not currently resident. A swap file is flexible to create and resize; a separate swap partition or device can offer simpler dedicated configuration. Swap is not a substitute for sufficient RAM in a latency-sensitive database.

4. RAID Comparison

RAID level	Minimum disks	Usable capacity for 6 × 4 TB	Fault tolerance / evaluation
RAID 0	2	24 TB	No redundancy; failure of one disk can lose the array.
RAID 1	2	Typically 12 TB for three	Can survive one disk failure in each mirror

		mirrored pairs	pair; capacity is reduced.
RAID 5	3	(6−1) × 4 = 20 TB	Distributed parity; survives one disk failure.
RAID 10	4	12 TB for six disks arranged as three mirror pairs	High performance and redundancy; can survive multiple failures if they are in different mirror pairs.

The capacities use decimal TB and assume equal-sized 4 TB disks, with no allowance for formatting overhead or vendor-specific usable capacity.

5. RAID 6 Alternative

For six 4 TB disks, RAID 6 usable capacity is (6−2) × 4 TB = 16 TB. RAID 6 can survive two disk failures. Writes require updating two parity values, so write processing is more complex than RAID 5. It is useful when protection against a second disk failure during rebuild is important.

6. Linux Commands and Evidence

```
# Display disks, partitions and mount points
lsblk

# Display partition tables (may require administrator privileges)
sudo fdisk -l

# Display swap configuration
swapon --show
free -h

# Example: show file-system usage
df -h
```

7. Recommendation

For a critical hospital records server, RAID 10 is a strong choice when performance and fast recovery are priorities and the reduced usable capacity is acceptable. RAID 6 is an alternative when higher usable capacity and tolerance of two disk failures are more important. RAID is not a backup; independent backups and tested recovery procedures are still required.

For 6 disks of 4 TB, the nominal usable capacities are RAID 0: 24 TB, RAID 1 (three mirrored pairs): 12 TB, RAID 5: 20 TB, RAID 10: 12 TB and RAID 6: 16 TB.

Q7 – DESIGN A MEMORY-MANAGEMENT PLAN FROM FIRST PRINCIPLES (10 MARKS)

Problem Context

A smart factory uses edge devices with 2 GB RAM and no swap disk. Each device runs a real-time control task that must never be delayed by a page fault, a machine-learning task requiring 1.5 GB and a logging task. The design must allow future tasks while protecting the real-time deadline.

Stage 1 – Interpret the Problem (2 Marks)

Task	Memory / timing need	Page-fault tolerance	Priority
Real-time control	Small, bounded memory footprint; strict response deadline.	Must not incur a page fault during its critical execution.	Highest
Machine-learning	Approximately 1.5 GB working memory; throughput-oriented.	Can tolerate some faults, but frequent faults reduce throughput.	Medium
Logging	Buffers and storage I/O; should not block control loop.	Can tolerate buffering and deferred writes.	Lower

The 2 GB RAM budget must also include the operating system, drivers, shared libraries, buffers and safety margin. Therefore, the machine-learning task cannot simply be assigned the full remaining 1.5 GB without accounting for system overhead.

Stage 2 – Compare Candidate Mechanisms (3 Marks)

Mechanism	How it works	Suitability for control task	Suitability for ML and logging
Contiguous allocation	Each task receives one continuous memory region.	Predictable placement, but allocation and growth can be difficult.	May waste space and make dynamic growth difficult.
Paging	Virtual pages map to physical frames.	Can introduce page faults and translation overhead; lock critical pages in RAM.	Flexible and supports virtual memory.
Segmentation	Memory is divided into logical units such as code, data and stack.	Useful for logical protection, but external fragmentation can occur.	Represents logical regions clearly.
Demand paging	Pages are loaded only when first accessed.	Unsuitable for a strict no-page-fault critical loop unless all needed pages are preloaded and locked.	Useful for larger workloads when active working set fits RAM.

Memory-Locking Strategy

On a Linux-based edge device, the control task should allocate its required memory before entering the real-time loop, touch the pages to make them resident and use mlock()/mlockall() where permitted. The critical loop should avoid dynamic allocation, blocking file I/O and operations that can fault. The system must reserve enough physical memory for the OS and other tasks. Memory locking reduces paging risk but does not by itself guarantee hard real-time behaviour.

Stage 3 – Implement and Verify (3 Marks)

The following Linux C example allocates a memory region, locks it, touches each page and measures the execution time of a simple loop. Run both locked and unlocked versions and record the maximum observed delay. The measurement is not a formal proof of a worst-case execution-time bound.

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/mman.h>
#include <time.h>
#include <unistd.h>

static long long ns_diff(struct timespec a, struct timespec b)
{
    return (b.tv_sec - a.tv_sec) * 1000000000LL +
           (b.tv_nsec - a.tv_nsec);
}

int main(int argc, char **argv)
{
    size_t bytes = 1024 * 1024;
    int lock_memory = (argc > 1 && strcmp(argv[1], "lock") == 0);
    volatile unsigned char *mem = malloc(bytes);
    if (!mem) { perror("malloc"); return 1; }

    long page = sysconf(_SC_PAGESIZE);
    for (size_t i = 0; i < bytes; i += (size_t)page)
        mem[i] = 1; /* touch pages before timing */

    if (lock_memory && mlock((const void *)mem, bytes) != 0) {
        perror("mlock");
        free((void *)mem);
        return 1;
    }

    struct timespec start, end;
    volatile unsigned long sum = 0;
    clock_gettime(CLOCK_MONOTONIC, &start);
    for (size_t i = 0; i < bytes; i += 64)
        sum += mem[i];
    clock_gettime(CLOCK_MONOTONIC, &end);

    printf("mode=%s elapsed_ns=%lld sum=%lu\n",
```

```
            lock_memory ? "locked" : "unlocked",
            ns_diff(start, end), sum);

    if (lock_memory) munlock((const void *)mem, bytes);
    free((void *)mem);
    return 0;
}
```

Compile with gcc -std=c11 -Wall -Wextra memory_lock.c -o memory_lock. Run ./memory_lock and ./memory_lock lock. The mlock call may require sufficient RLIMIT_MEMLOCK privileges. Record any permission error instead of claiming the memory was locked.

Page-Replacement Simulation

A simple page-replacement simulator can be used to model the memory demand of the control, machine-learning and logging tasks. Keep the control task's pages resident in the model and allocate the remaining frames to the other tasks. Record the page-fault count and confirm that the simulated control references do not fault.

Stage 4 – Defend the Design (2 Marks)

The recommended plan is to reserve and lock the control task's code, stack and working data before the system begins production. Run the machine-learning task with a controlled memory budget and schedule it so it cannot consume the control task's reserved resources. Use buffered, lower-priority logging. Monitor free memory, page faults and task response times.

If a Fourth Task of 500 MB Is Added

A fourth 500 MB task cannot be added by simply assuming that 2 GB is sufficient. The system must calculate the OS and driver footprint, the real-time reservation, the machine-learning working set, logging buffers and safety margin. If the total exceeds physical RAM, reduce or stream the ML working set, move non-critical processing to a server, or upgrade device memory. Do not allow the new task to consume the control task's reserved pages.

Remaining Limitation

Memory locking prevents selected pages from being paged out, but it does not eliminate delays caused by interrupts, device drivers, CPU contention, cache misses or non-real-time kernel behaviour. A hard real-time guarantee requires a suitable real-time OS/kernel, careful hardware design and worst-case timing analysis.

Alternative: Real-Time OS Without Virtual Memory

A real-time OS without virtual memory can provide predictable memory behaviour and avoid demand-paging faults. The control task can have a bounded, statically allocated memory region. However, the machine-learning task has less flexibility to use a large virtual address space, and adding future tasks requires careful static memory planning. Linux with locked memory is easier for complex applications but requires disciplined configuration and does not automatically provide a hard deadline guarantee.

Measurement Table – Fill After Execution

Test	Unlocked	Locked	Observation
Maximum observed loop time	_____	_____	_____
Average loop time	_____	_____	_____
Page faults	_____	_____	_____
Memory allocated / locked	_____	_____	_____

OVERALL CONCLUSION

This assignment evaluated memory-management and storage mechanisms for different workloads. Linking and loading determine how program modules and libraries are combined and mapped into memory. Dynamic linking supports independent library updates, while static linking can provide a self-contained executable. Paging enables non-contiguous physical allocation and virtual memory, while segmentation represents logical program regions and supports protection. Demand paging allows programs to use a virtual address space larger than RAM, but excessive page faults can cause thrashing. Page-replacement policies influence fault rates, with LRU and clock-style methods generally offering better locality behaviour than FIFO. Disk management and RAID selection must balance capacity, performance and fault tolerance. For the smart-factory edge device, the real-time control task should have a reserved and locked working set, while machine-learning and logging tasks should operate within controlled memory budgets. Final decisions should be supported by actual program execution, measurements and screenshots from the student's Linux environment.

REFERENCES

[1] A. Silberschatz, P. B. Galvin and G. Gagne, Operating System Concepts, 10th ed., Wiley, 2018.

[2] A. S. Tanenbaum and H. Bos, Modern Operating Systems, 4th ed., Pearson, 2015.

[3] R. Love, Linux Kernel Development, 3rd ed., Addison-Wesley, 2010.

[4] Linux man-pages project, “mmap(2), mlock(2), fork(2), dlopen(3), read(2),” Linux manual pages. Available: https://man7.org/linux/man-pages/

[5] The Open Group, “POSIX.1 System Interfaces,” The Open Group Base Specifications. Available: https://pubs.opengroup.org/onlinepubs/9699919799/

[6] Linux Kernel Documentation, “Memory Management,” kernel.org documentation. Available: https://docs.kernel.org/admin-guide/mm/

[7] GNU Project, “GNU Compiler Collection (GCC) Documentation,” GCC online documentation. Available: https://gcc.gnu.org/onlinedocs/

[8] Linux man-pages project, “proc(5) – process information pseudo-filesystem,” Linux manual pages. Available: https://man7.org/linux/man-pages/man5/proc.5.html

[9] NVM Express, “NVM Express Base Specification,” NVM Express, specification resources. Available: https://nvmexpress.org/specifications/

[10] SNIA, “Storage Networking Industry Association Dictionary and Technical Resources,” SNIA. Available: https://www.snia.org/education/online-dictionary